Kubernates

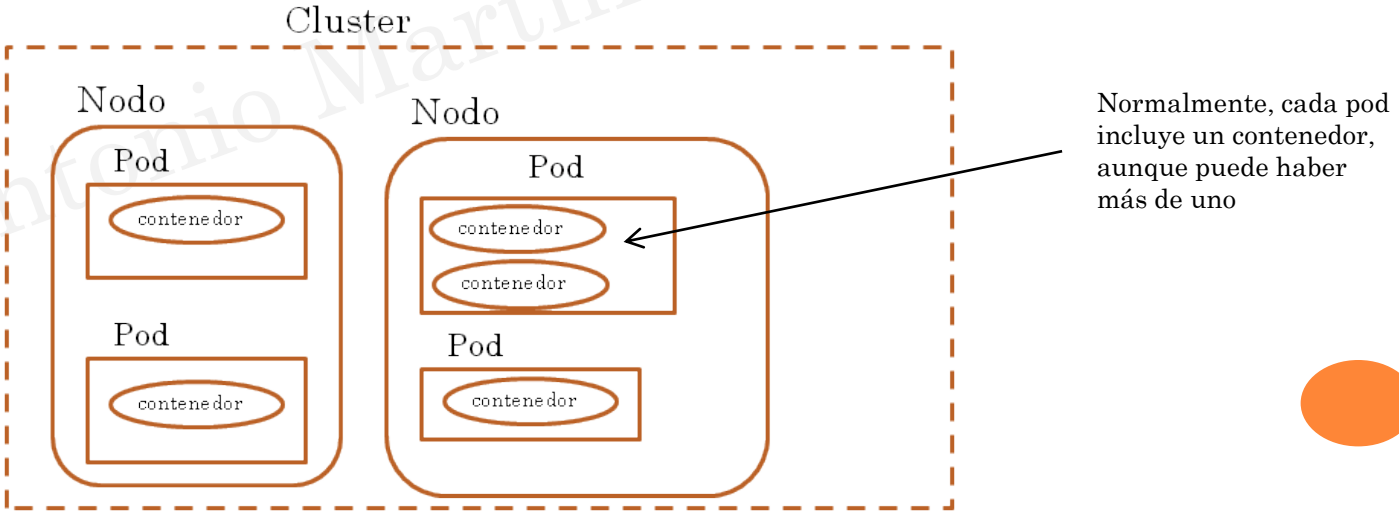
¿Qué es kubernates?

- Sistema de orquestación de contenedores que administra el despliegue de los mismos en un entorno de producción
- Los contenedores se organizan en pods, que se ejecutan en nodos dentro de un clúster kubernates
- Permite el escalado de aplicaciones y proporciona alta disponibilidad al sistema
- El despliegue y gestión de aplicaciones se realiza a través del archivo deployment.yaml

Antonio Martín Sierra

Arquitectura

- **Nodo: Máquina física o virtual donde se ejecuta kubernates**
- **Pod: Parte dentro de un nodo donde se despliegan aplicaciones, normalmente contenedores**
- **Cluster: Agrupación de nodos para alta disponibilidad**



Antonio Martín Sierra

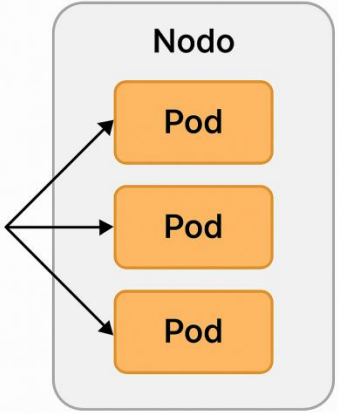
Diferencia Nodo/Pod

Nodo:

- Máquina física o virtual.
- Servidor dentro del cluster.
- Cada nodo tiene su CPU, memoria y agente Kubelet

Pod:

- Unidad mínima de despliegue.
- Ejecuta los contenedores.
- Permite aislar y gestionar contenedores de forma estructurada



Antonio Martín Sierra

INSTALACIÓN

- **Para trabajar con Kubernates en la máquina local utilizaremos minikube, que se puede descargar en:**

  https://minikube.sigs.k8s.io/docs/start

- **En windows descargamos installer.exe y lo ejecutamos para proceder a la instalación.**

- **Se crea un cluster vacío**

- **Una vez instalado, podemos iniciar minikube mediante:**

  >minikube start

Antonio Martín Sierra

Comandos básicos I

- **Listar pods existentes:**

  >kubectl get pod

  >kubectl get pod -A ← Igual que el anterior, pero muestra también el namespace

  >kubectl get pod -o wide ← Muestra en que no do está cada pod

- **Listar deployments:**

  >kubectl get deployments

- **Listar servicios**

  >kubectl get services

- **Mostrar logs de un pod**

  >kubectl logs nombre_pod

Antonio Martín Sierra

Comandos básicos II

- **Eliminar un pod:**

  >kubectl delete pod nombre_pod

- **Eliminar deployment:**

  >kubectl delete deployment nombre_deployment

- **Eliminar service**

  >kubectl delete service nombre_service

- **Para eliminar todos pods, deployments, services:**

  >kubectl delete pods --all
  >kubectl delete deployments --all
  >kubectl delete services --all

- **Para borrar todos los componentes de minikube**

  >minikube delete

Antonio Martín Sierra

El archivo deployment.yaml

- **Define la información para el despliegue de una aplicación en kubernates. Su estructura es:**

Nombre del despliegue

Número de réplicas (pods) que se van a crear

Indica qué pods están gestionados por el deployment

Imagen docker a partir de la cual se creará el contenedor

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: buscador-deployment
spec:
  replicas: 1
  selector:
    matchLabels:
      app: buscador
  template:
    metadata:
      labels:
        app: buscador
    spec:
      containers:
        - name: buscador-container
          image: imgbuscador  # Usa la imagen Docker local
          imagePullPolicy: Never
          ports:
            - containerPort: 8000  # Puerto dentro del contenedor
```

Etiqueta asignada a los pods

Puerto sobre el que escucha peticiones el microservicio

El archivo service.yaml

- Un service proporciona un punto de acceso a un conjunto de pods que exponen las mismas aplicaciones
- Se define a través del archivo service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: buscador-service
spec:
  selector:
    app: buscador
  type: NodePort  # Hace accesible el servicio desde fuera del clúster
  ports:
    - protocol: TCP
      port: 8000      # Puerto del servicio en Kubernetes
      targetPort: 8000 # Puerto del contenedor
      nodePort: 30001   # Puerto en la máquina kubernates
```

Indica los pods que se deben enrutar

No significa que sea el puerto de la máquina física

Despliegue de un microservicio

1. **Se debe hacer accesible la imagen docker en kubernates**

   >minikube image load imgbuscador

2. **Se aplican los archivos de despliegue:**

   >kubectl apply -f deployment.yaml
   >kubectl apply -f service.yaml

3. **Se obtiene la dirección de acceso al servicio a través de la máquina física:**

   >minikube service buscador-service --url

Si lanzamos peticiones a esa URL balancea entre los pods, aunque con postman no se aprecia porque mantiene conexión viva. Probar con curl:

>curl --no-keepalive direccion

O hacer esto para lanzar 10 peticiones o observar balanceo:

>for /L %i in (1,1,10) do curl http://127.0.0.1:63494

Antonio Martín Sierra

Acceso a través de máquina física

- **En minikube, el servicio debe estar desplegado como loadbalancer, no como nodePort.**

- **Se debe crear un tunel, bien como se indicó en el punto 3 de la diapositiva anterior (genera un puerto distinto cada vez), o mediante:**

```
>minikube tunnel
```

- **Se puede acceder ahora al microservicio con:**

http://ip_externa:port

se puede consultar la ip externa con:
kubectl get svc

el port indicado en la sección ports del service.yaml

Antonio Martín Sierra

Integración Spring Boot y Kubernates

Starter Kubernates-client

- Utilizado por microservicios Spring Boot que van a consumir servicios desplegados en un cluster kubernates

- Entre otras cosas, permite descubrir otros servicios desplegados en un cluster sin necesidad de Eureka

- Se deberá incluir la siguiente dependencia en pom.xml:

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-kubernetes-client</artifactId>
</dependency>
```

io Martín Sierra

Compatibilidad de versiones

- **Actualmente, he conseguido que el starter anterior funcione con spring cloud 2023.0.1 en la versión spring boot 3.3.2**

- **Por tanto, se incluirá además en el pom.xml:**

```
<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>org.springframework.cloud</groupId>
      <artifactId>spring-cloud-dependencies</artifactId>
      <version>2023.0.1</version>
      <type>pom</type>
      <scope>import</scope>
    </dependency>
  </dependencies>
</dependencyManagement>
```

Descubrimiento de servicios

- **Para descubrir servicios en el cluster, el microservicio cliente utilizará http://nombre_servicio:**

```
public class FormacionServiceImpl implements FormacionService {
    String urlBase="http://alumnos-service/";
    @Autowired
    private RestClient restClient;
    @Override
    public List<Estudiante> estudiantesNota(double notaMin) {
        return Arrays.stream(restClient
                .get()
                .uri(urlBase+"alumnos")
                :                                :
        }
}
```

- **No se requiere ninguna anotación especial, tipo @LoadBalanced en la creación de RestClient**

Permisos de acceso a cluster

- Las aplicaciones kubernates-client desplegadas en un cluster intentan listar los servicios y endpoints para poder acceder a ellos.

- Por defecto, kubernates rechazará esta operación al no tener permisos la aplicación para ello.

- Serán necesario definir un rol con permisos para esta tarea y asociarlo al ServiceAccount del pod donde se despliegará el microservicio cliente

Antonio Martín Gerna

Archivo role.yaml

recursos del namespace a los que se tendrá acceso

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: default  # o el namespace donde está el pod
  name: rol-formacion
rules:
- apiGroups: [""]
  resources: ["services","endpoints"]
  verbs: ["get", "list", "watch"]
```

- **El rol se aplica antes del despliegue del servicio:**

  >kubectl apply -f role.yaml

Archivo rolebinding.yaml

- **Asocia el rol al ServiceAccount:**

nombre del ServiceAccount

```
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: formacion-binding
  namespace: default
subjects:
- kind: ServiceAccount
  name: default   # o el ServiceAccount que use el pod
  namespace: default
roleRef:
  kind: Role
  name: rol-formacion
  apiGroup: rbac.authorization.k8s.io
```

- **El binding se aplica antes del despliegue del servicio:**

>kubectl apply -f rolebinding.yaml

Despliegue del servicio cliente

- **Se seguirán los pasos descritos para el despliegue de un microservicio en el cluster.**

- **Un detalle importante: Para que el microservicio cliente pueda acceder al servicio sin necesidad de indicar su puerto, este servicio debe estar escuchando en el puerto 80:**

service.yaml del microservicio que va a ser consumido en el cluster

```yaml
apiVersion: v1
kind: Service
metadata:
  name: alumnos-service
spec:
  selector:
    app: alumnos
  type: NodePort
  ports:
    - protocol: TCP
      port: 80      # Puerto del servicio en Kubernetes
      targetPort: 8001 # Puerto del contenedor
      nodePort: 30002   # Puerto en la máquina física
```